

DNS



Async DNS resolver for ReactPHP.

The main point of the DNS component is to provide async DNS resolution. However, it is really a toolkit for working with DNS messages, and could easily be used to create a DNS server.

Table of contents

- Basic usage
- Caching
 - Custom cache adapter
- ResolverInterface
 - resolve()
 - resolveAll()
- Advanced usage
 - UdpTransportExecutor
 - TcpTransportExecutor
 - SelectiveTransportExecutor
 - HostsFileExecutor
- Install
- Tests
- License
- References

Basic usage

The most basic usage is to just create a resolver through the resolver factory. All you need to give it is a nameserver, then you can start resolving names, baby!

```
$config = React\Dns\Config\Config::loadSystemConfigBlocking();
if (!$config->nameservers) {
    $config->nameservers[] = '8.8.8.8';
}

$factory = new React\Dns\Resolver\Factory();
$dns = $factory->create($config);

$dns->resolve('igor.io')->then(function ($ip) {
    echo "Host: $ip\n";
});
```

See also the first example.

The `Config` class can be used to load the system default config. This is an operation that may access the filesystem and block. Ideally, this method should

thus be executed only once before the loop starts and not repeatedly while it is running. Note that this class may return an *empty* configuration if the system config can not be loaded. As such, you'll likely want to apply a default nameserver as above if none can be found.

Note that the factory loads the hosts file from the filesystem once when creating the resolver instance. Ideally, this method should thus be executed only once before the loop starts and not repeatedly while it is running.

But there's more.

Caching

You can cache results by configuring the resolver to use a `CachedExecutor`:

```
$config = React\Dns\Config\Config::loadSystemConfigBlocking();
if (!$config->nameservers) {
    $config->nameservers[] = '8.8.8.8';
}

$factory = new React\Dns\Resolver\Factory();
$dns = $factory->createCached($config);

$dns->resolve('igor.io')->then(function ($ip) {
    echo "Host: $ip\n";
});

...

$dns->resolve('igor.io')->then(function ($ip) {
    echo "Host: $ip\n";
});
```

If the first call returns before the second, only one query will be executed. The second result will be served from an in memory cache. This is particularly useful for long running scripts where the same hostnames have to be looked up multiple times.

See also the third example.

Custom cache adapter

By default, the above will use an in memory cache.

You can also specify a custom cache implementing `CacheInterface` to handle the record cache instead:

```
$cache = new React\Cache\ArrayCache();
```

```
$factory = new React\Dns\Resolver\Factory();
$dns = $factory->createCached('8.8.8.8', null, $cache);
```

See also the wiki for possible cache implementations.

ResolverInterface

resolve()

The `resolve(string $domain): PromiseInterface<string>` method can be used to resolve the given \$domain name to a single IPv4 address (type A query).

```
$resolver->resolve('reactphp.org')->then(function ($ip) {
    echo 'IP for reactphp.org is ' . $ip . PHP_EOL;
});
```

This is one of the main methods in this package. It sends a DNS query for the given \$domain name to your DNS server and returns a single IP address on success.

If the DNS server sends a DNS response message that contains more than one IP address for this query, it will randomly pick one of the IP addresses from the response. If you want the full list of IP addresses or want to send a different type of query, you should use the `resolveAll()` method instead.

If the DNS server sends a DNS response message that indicates an error code, this method will reject with a `RecordNotFoundException`. Its message and code can be used to check for the response code.

If the DNS communication fails and the server does not respond with a valid response message, this message will reject with an `Exception`.

Pending DNS queries can be cancelled by cancelling its pending promise like so:

```
$promise = $resolver->resolve('reactphp.org');

$promise->cancel();
```

resolveAll()

The `resolveAll(string $host, int $type): PromiseInterface<array>` method can be used to resolve all record values for the given \$domain name and query \$type.

```
$resolver->resolveAll('reactphp.org', Message::TYPE_A)->then(function ($ips) {
    echo 'IPv4 addresses for reactphp.org ' . implode(' ', $ips) . PHP_EOL;
});

$resolver->resolveAll('reactphp.org', Message::TYPE_AAAA)->then(function ($ips) {
    echo 'IPv6 addresses for reactphp.org ' . implode(' ', $ips) . PHP_EOL;
});
```

This is one of the main methods in this package. It sends a DNS query for the given \$domain name to your DNS server and returns a list with all record values on success.

If the DNS server sends a DNS response message that contains one or more records for this query, it will return a list with all record values from the response. You can use the `Message::TYPE_*` constants to control which type of query will be sent. Note that this method always returns a list of record values, but each record value type depends on the query type. For example, it returns the IPv4 addresses for type `A` queries, the IPv6 addresses for type `AAAA` queries, the hostname for type `NS`, `CNAME` and `PTR` queries and structured data for other queries. See also the `Record` documentation for more details.

If the DNS server sends a DNS response message that indicates an error code, this method will reject with a `RecordNotFoundException`. Its message and code can be used to check for the response code.

If the DNS communication fails and the server does not respond with a valid response message, this message will reject with an `Exception`.

Pending DNS queries can be cancelled by cancelling its pending promise like so:

```
$promise = $resolver->resolveAll('reactphp.org', Message::TYPE_AAAA);

$promise->cancel();
```

Advanced Usage

UdpTransportExecutor

The `UdpTransportExecutor` can be used to send DNS queries over a UDP transport.

This is the main class that sends a DNS query to your DNS server and is used internally by the `Resolver` for the actual message transport.

For more advanced usages one can utilize this class directly. The following example looks up the IPv6 address for `igor.io`.

```
$executor = new UdpTransportExecutor('8.8.8.8:53');

$executor->query(
    new Query($name, Message::TYPE_AAAA, Message::CLASS_IN)
)->then(function (Message $message) {
    foreach ($message->answers as $answer) {
        echo 'IPv6: ' . $answer->data . PHP_EOL;
    }
}, 'printf');
```

See also the fourth example.

Note that this executor does not implement a timeout, so you will very likely want to use this in combination with a `TimeoutExecutor` like this:

```
$executor = new TimeoutExecutor(  
    new UdpTransportExecutor($nameserver),  
    3.0  
);
```

Also note that this executor uses an unreliable UDP transport and that it does not implement any retry logic, so you will likely want to use this in combination with a `RetryExecutor` like this:

```
$executor = new RetryExecutor(  
    new TimeoutExecutor(  
        new UdpTransportExecutor($nameserver),  
        3.0  
    )  
);
```

Note that this executor is entirely async and as such allows you to execute any number of queries concurrently. You should probably limit the number of concurrent queries in your application or you're very likely going to face rate limitations and bans on the resolver end. For many common applications, you may want to avoid sending the same query multiple times when the first one is still pending, so you will likely want to use this in combination with a `CoopExecutor` like this:

```
$executor = new CoopExecutor(  
    new RetryExecutor(  
        new TimeoutExecutor(  
            new UdpTransportExecutor($nameserver),  
            3.0  
        )  
    )  
);
```

Internally, this class uses PHP's UDP sockets and does not take advantage of react/datagram purely for organizational reasons to avoid a cyclic dependency between the two packages. Higher-level components should take advantage of the Datagram component instead of reimplementing this socket logic from scratch.

TcpTransportExecutor

The `TcpTransportExecutor` class can be used to send DNS queries over a TCP/IP stream transport.

This is one of the main classes that send a DNS query to your DNS server.

For more advanced usages one can utilize this class directly. The following example looks up the IPv6 address for `reactphp.org`.

```
$executor = new TcpTransportExecutor('8.8.8.8:53');

$executor->query(
    new Query($name, Message::TYPE_AAAA, Message::CLASS_IN)
)->then(function (Message $message) {
    foreach ($message->answers as $answer) {
        echo 'IPv6: ' . $answer->data . PHP_EOL;
    }
}, 'printf');
```

See also example #92.

Note that this executor does not implement a timeout, so you will very likely want to use this in combination with a `TimeoutExecutor` like this:

```
$executor = new TimeoutExecutor(
    new TcpTransportExecutor($nameserver),
    3.0
);
```

Unlike the `UdpTransportExecutor`, this class uses a reliable TCP/IP transport, so you do not necessarily have to implement any retry logic.

Note that this executor is entirely async and as such allows you to execute queries concurrently. The first query will establish a TCP/IP socket connection to the DNS server which will be kept open for a short period. Additional queries will automatically reuse this existing socket connection to the DNS server, will pipeline multiple requests over this single connection and will keep an idle connection open for a short period. The initial TCP/IP connection overhead may incur a slight delay if you only send occasional queries – when sending a larger number of concurrent queries over an existing connection, it becomes increasingly more efficient and avoids creating many concurrent sockets like the UDP-based executor. You may still want to limit the number of (concurrent) queries in your application or you may be facing rate limitations and bans on the resolver end. For many common applications, you may want to avoid sending the same query multiple times when the first one is still pending, so you will likely want to use this in combination with a `CoopExecutor` like this:

```
$executor = new CoopExecutor(
    new TimeoutExecutor(
        new TcpTransportExecutor($nameserver),
        3.0
    )
);
```

Internally, this class uses PHP's TCP/IP sockets and does not take advantage of react/socket purely for organizational reasons to avoid

a cyclic dependency between the two packages. Higher-level components should take advantage of the Socket component instead of reimplementing this socket logic from scratch.

SelectiveTransportExecutor

The `SelectiveTransportExecutor` class can be used to Send DNS queries over a UDP or TCP/IP stream transport.

This class will automatically choose the correct transport protocol to send a DNS query to your DNS server. It will always try to send it over the more efficient UDP transport first. If this query yields a size related issue (truncated messages), it will retry over a streaming TCP/IP transport.

For more advanced usages one can utilize this class directly. The following example looks up the IPv6 address for `reactphp.org`.

```
$executor = new SelectiveTransportExecutor($udpExecutor, $tcpExecutor);

$executor->query(
    new Query($name, Message::TYPE_AAAA, Message::CLASS_IN)
)->then(function (Message $message) {
    foreach ($message->answers as $answer) {
        echo 'IPv6: ' . $answer->data . PHP_EOL;
    }
}, 'printf');
```

Note that this executor only implements the logic to select the correct transport for the given DNS query. Implementing the correct transport logic, implementing timeouts and any retry logic is left up to the given executors, see also `UdpTransportExecutor` and `TcpTransportExecutor` for more details.

Note that this executor is entirely async and as such allows you to execute any number of queries concurrently. You should probably limit the number of concurrent queries in your application or you're very likely going to face rate limitations and bans on the resolver end. For many common applications, you may want to avoid sending the same query multiple times when the first one is still pending, so you will likely want to use this in combination with a `CoopExecutor` like this:

```
$executor = new CoopExecutor(
    new SelectiveTransportExecutor(
        $datagramExecutor,
        $streamExecutor
    )
);
```

HostsFileExecutor

Note that the above `UdpTransportExecutor` class always performs an actual DNS query. If you also want to take entries from your hosts file into account, you may use this code:

```
$hosts = \React\Dns\Config\HostsFile::loadFromPathBlocking();

$executor = new UdpTransportExecutor('8.8.8.8:53');
$executor = new HostsFileExecutor($hosts, $executor);

$executor->query(
    new Query('localhost', Message::TYPE_A, Message::CLASS_IN)
);
```

Install

The recommended way to install this library is through Composer. New to Composer?

This project follows SemVer. This will install the latest supported version:

```
composer require react/dns:^1.13
```

See also the CHANGELOG for details about version upgrades.

This project aims to run on any platform and thus does not require any PHP extensions and supports running on legacy PHP 5.3 through current PHP 8+ and HHVM. It's *highly recommended to use the latest supported PHP version* for this project.

Tests

To run the test suite, you first need to clone this repo and then install all dependencies through Composer:

```
composer install
```

To run the test suite, go to the project root and run:

```
vendor/bin/phpunit
```

The test suite also contains a number of functional integration tests that rely on a stable internet connection. If you do not want to run these, they can simply be skipped like this:

```
vendor/bin/phpunit --exclude-group internet
```

License

MIT, see LICENSE file.

References

- RFC 1034 Domain Names - Concepts and Facilities
- RFC 1035 Domain Names - Implementation and Specification